

Introducción

Para cubrir estas necesidades, Flutter provee un modelo de superposición basado en capas. En esta sesión analizaremos a detalle la gestión de restricciones y dimensionamiento en contenedores superpuestos con **Stack**, la ubicación precisa en coordenadas absolutas con **Positioned**, la alineación geográfica y vectorial con **Align**, y la optimización de vistas persistentes mediante **IndexedStack**.

El modelo de capas en Flutter opera apilando widgets de atrás hacia adelante en el eje Z de acuerdo con su orden de declaración.

```

+-----+
--+
|
|
+-----+
--+

|
+-----+-----+
|               |               |
v               v               v
+-----+       +-----+       +-----+
---+           ---+           ---+
| MANEJO DE CAPAS | | POSICIONAMIENTO | | NUEVA CAPA DE
|               | |               | |
| Y RESTRICCION  | | ABSOLUTO Y REL   | | PERSISTENCIA
|               | |               | |
| (STACK)        | | (POSITION/ALIGN) | |
| (INDEXEDSTACK) | |               | |
+-----+       +-----+       +-----+
---+           ---+           ---+
| • Children (Z)  | | • Top/Bottom/Left| | • index
|               | |               | |
| • Alignment     | | • Positioned.fill| | • Children
|               | |               | |

```

• StackFit	• Alignment	• Mantiene
state		
• ClipBehavior	• Alignment(x,y)	• Single child
v		
+-----+	+-----+	+-----
---+		

1. Gestión de Capas y Restricciones: **Stack**

El widget **Stack** permite colocar múltiples componentes unos sobre otros. La regla fundamental de dimensionamiento de un **Stack** establece que **su tamaño se adapta al hijo no posicionante (*non-positioned*) más grande**, salvo que se apliquen restricciones explícitas del padre.

- **alignment**: Define la alineación por defecto para todos los hijos **no posicionados** dentro de las fronteras del **Stack** (ej. **Alignment.center**, **AlignmentDirectional.topStart**).
- **fit (Reglas de Restricción - StackFit)**:
 - **StackFit.loose** (*predeterminado*): Los hijos no posicionados pueden tomar cualquier dimensión dentro de las restricciones máximas permitidas por el padre.
 - **StackFit.expand**: Fuerza a los hijos no posicionados a tomar el tamaño máximo disponible permitida por el **Stack**.
 - **StackFit.passthrough**: Transmite las restricciones entrantes del padre del **Stack** directamente a sus hijos sin alteración.
- **clipBehavior**: Define el comportamiento de recorte si un hijo sobresale del marco físico del **Stack**.
- **Clip.none**: Permite renderizar elementos fuera de la caja contenedora (ideal para insignias y botones flotantes superpuestos).
- **Clip.hardEdge** / **Clip.antiAlias**: Recorta estrictamente todo elemento que exceda el perímetro.

2. Posicionamiento Absoluto y Relativo: **Positioned** y **Align**

- **Positioned**:
 - Debe ser hijo **directo** de un **Stack**. Un hijo envuelto en **Positioned** se considera un *positioned child* y no afecta el tamaño global del **Stack**.
 - Define la ubicación exacta en píxeles desde las fronteras usando **top**, **bottom**, **left**, y **right**.
 - **Positioned.fill**: Shortcut para expandir el hijo cubriendo las cuatro esquinas (**top: 0**, **bottom: 0**, **left: 0**, **right: 0**).
- **Align**:
 - Posiciona a su hijo dentro del espacio disponible mediante coordenadas de alineación relativas.

- **alignment**: Acepta constantes como `Alignment.bottomRight` o un punto vectorial `Alignment(x, y)` donde $(0, 0)$ es el centro exacto, $(-1.0, -1.0)$ la esquina superior izquierda y $(1.0, 1.0)$ la esquina inferior derecha.

3. Conmutación Persistente de Vistas: `IndexedStack`

`IndexedStack` es una subclase especializada de `Stack` que solo muestra un único hijo a la vez, según la propiedad `index`.

- **Ventaja Arquitectónica**: A diferencia de un operador ternario o un bloque `switch` convencional que destruye y vuelve a crear los widgets al cambiar de vista, `IndexedStack` **mantiene instanciados en memoria todos sus hijos**.
- **Preservación de Estado**: Permite conservar el estado interno (`State`), como la posición de desplazamientos en listas o textos ingresados en formularios, al conmutar entre diferentes pestañas o secciones de la interfaz.

Ejemplo Completo con Código Fuente: Consola de Monitoreo con Vista Persistente y Superposición

El siguiente código implementa una pantalla de gestión que utiliza un `IndexedStack` para cambiar de pestaña conservando el estado, combinado con un `Stack`, `Positioned`, `Align` y `Clip.none` para construir tarjetas con badges e indicadores de notificación.

Código Fuente Explicado (`lib/main.dart`)

```
import 'package:flutter/material.dart';

void main() {
  runApp(const DemoPosicionamientoAvanzadoApp());
}

/// Widget Raíz
class DemoPosicionamientoAvanzadoApp extends StatelessWidget {
  const DemoPosicionamientoAvanzadoApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Posicionamiento Avanzado',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: const PantallaConsolaPrincipal(),
    );
  }
}
```

```
}

/// Pantalla Principal con Navegación mediante IndexedStack
class PantallaConsolaPrincipal extends StatefulWidget {
  const PantallaConsolaPrincipal({super.key});

  @override
  State<PantallaConsolaPrincipal> createState() =>
    _PantallaConsolaPrincipalState();
}

class _PantallaConsolaPrincipalState
  extends State<PantallaConsolaPrincipal> {
  int _indiceSeleccionado = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Consola de Control - Capas y Posicionamiento'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),

      // PERSISTENCIA DE ESTADO CON INDEXEDSTACK
      body: IndexedStack(
        index: _indiceSeleccionado,
        children: const [
          VistaTarjetasSuperpuestas(), // Índice 0
          VistaListaPersistente(),     // Índice 1
        ],
      ),

      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _indiceSeleccionado,
        onTap: (nuevoIndice) {
          setState(() {
            _indiceSeleccionado = nuevoIndice;
          });
        },
        items: const [
          BottomNavigationBarItem(
            icon: Icon(Icons.layers),
            label: 'Capas / Stack',
          ),
          BottomNavigationBarItem(
            icon: Icon(Icons.history),
            label: 'Estado Persistente',
          ),
        ],
      ),
    );
  }
}
```

```

/// Vista 1: Demostración de Stack, Positioned, Align y Clip.none
class VistaTarjetasSuperpuestas extends StatelessWidget {
  const VistaTarjetasSuperpuestas({super.key});

  @override
  Widget build(BuildContext context) {
    return SingleChildScrollView(
      padding: const EdgeInsets.all(24.0),
      child: Column(
        children: [
          const Text(
            'Manejo de Capas, Coordenadas y Recortado',
            style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
          ),
          const SizedBox(height: 30),

          // EJEMPLO 1: TARJETA DE PERFIL CON BADGE SOBRESALIENTE
          Center(
            child: SizedBox(
              width: 320,
              height: 180,
              // STACK PRINCIPAL
              child: Stack(
                clipBehavior: Clip.none, // Permite que el badge sobresalga del
                fit: StackFit.loose,
                children: [
                  // Capa 1: Fondo de la tarjeta (Hijo No Posicionado)
                  Container(
                    width: double.infinity,
                    height: double.infinity,
                    padding: const EdgeInsets.all(16.0),
                    decoration: BoxDecoration(
                      color: Colors.white,
                      borderRadius: BorderRadius.circular(16),
                      boxShadow: const [
                        BoxShadow(
                          color: Colors.black12,
                          blurRadius: 10,
                          offset: Offset(0, 4),
                        ),
                      ],
                    ),
                  ),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: const [
                      Text(
                        'Servidor de Producción',
                        style: TextStyle(
                          fontSize: 18,
                          fontWeight: FontWeight.bold,
                        ),
                      ),

```

borde

```

    ),
    SizedBox(height: 6),
    Text(
      'IP: 192.168.1.250',
      style: TextStyle(color: Colors.grey),
    ),
    SizedBox(height: 12),
    Text(
      'Estado: Activo - Carga: 24%',
      style: TextStyle(
        color: Colors.green,
        fontWeight: FontWeight.w600,
      ),
    ),
  ],
),
),

// Capa 2: Icono de Marca de Agua Usando Align Vectorial
Align(
  alignment: const Alignment(0.85, 0.85), // Coordenada
cartesiana relativa
  child: Icon(
    Icons.dns,
    size: 64,
    color: Colors.grey.shade200,
  ),
),

// Capa 3: Insignia (Badge) Posicionada sobresaliendo en la
esquina
Positioned(
  top: -12,
  right: -12,
  child: Container(
    padding: const EdgeInsets.symmetric(
      horizontal: 12,
      vertical: 6,
    ),
    decoration: BoxDecoration(
      color: Colors.red.shade700,
      borderRadius: BorderRadius.circular(20),
      boxShadow: const [
        BoxShadow(
          color: Colors.black26,
          blurRadius: 4,
        ),
      ],
    ),
  child: const Row(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      Icon(Icons.warning, color: Colors.white, size: 14),
      SizedBox(width: 4),
    ],
  ),
),

```

```
Text(
  'CRÍTICO',
  style: TextStyle(
    color: Colors.white,
    fontSize: 10,
    fontWeight: FontWeight.bold,
  ),
),
],
),
),
),
),
// Capa 4: Barra de estado inferior con Positioned.fill + Align
Positioned.fill(
  child: Align(
    alignment: Alignment.bottomCenter,
    child: Container(
      height: 6,
      decoration: BoxDecoration(
        color: Theme.of(context).colorScheme.primary,
        borderRadius: BorderRadius.vertical(
          bottom: Radius.circular(16),
        ),
      ),
    ),
  ),
),
),
),
),
),
],
),
),
),
),
);
}
```

```

/// Vista 2: Demostración de Preservación de Estado con IndexedStack
class VistaListaPersistente extends StatefulWidget {
  const VistaListaPersistente({super.key});

  @override
  State<VistaListaPersistente> createState() => _VistaListaPersistenteState();
}

class _VistaListaPersistenteState extends State<VistaListaPersistente> {
  final TextEditingController _textController = TextEditingController();

  @override
  void dispose() {
    _textController.dispose();
    super.dispose();
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Padding(
    padding: const EdgeInsets.all(16.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Text(
          'Prueba de Preservación de Estado',
          style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
        ),
        const SizedBox(height: 8),
        const Text(
          'Escribe texto o desplaza la lista y cambia de pestaña. Gracias a
IndexedStack, la información y la posición no se destruyen.',
          style: TextStyle(fontSize: 12, color: Colors.grey),
        ),
        const SizedBox(height: 16),
        TextField(
          controller: _textController,
          decoration: const InputDecoration(
            labelText: 'Ingrese observaciones de auditoría',
            border: OutlineInputBorder(),
          ),
        ),
        const SizedBox(height: 16),
        Expanded(
          child: ListView.builder(
            itemCount: 30,
            itemBuilder: (context, index) {
              return ListTile(
                leading: CircleAvatar(child: Text('${index + 1}')),
                title: Text('Registro de auditoría #${index + 1}'),
                subtitle: const Text('Estado verificado sin alteración'),
              );
            },
          ),
        ),
      ],
    ),
  );
}

```

Prácticas Guiadas / Laboratorio

Práctica 1: Comprobación Experimental de Persistencia en **IndexedStack** vs Operador Condicional

1. Modifica la propiedad `body` de la pantalla principal en el archivo del proyecto para reemplazar el `IndexedStack` por una condición ternaria convencional:

```
body: _indiceSeleccionado == 0
  ? const VistaTarjetasSuperpuestas()
  : const VistaListaPersistente(),
```

2. Ejecuta la aplicación, navega a la pestaña "Estado Persistente", escribe un texto en el campo `TextField` y desplázate hacia el elemento 20 de la lista.
3. Cambia a la pestaña "Capas / Stack" y regresa inmediatamente a "Estado Persistente".
4. **Observación y Análisis:** Comprueba cómo con el operador ternario el texto desaparece y el scroll vuelve al inicio (el estado se destruyó). Restaura el `IndexedStack` y repite la prueba para validar cómo se preserva completamente el estado.

Práctica 2: Construcción de una Tarjeta de Producto con Botón Flotante Superpuesto

1. Crea un widget para representar una tarjeta de catálogo de productos usando un `Stack`.
2. Asigna una imagen o contenedor de fondo para la portada de la tarjeta.
3. Agrega un widget `Positioned` con coordenadas `bottom: -15` y `right: 15` conteniendo un `FloatingActionButton.small`.
4. Define la propiedad `clipBehavior: Clip.none` en el `Stack` para que el botón sobresalga visualmente del límite de la tarjeta sin recortarse.

Resumen y Puntos Clave

- **Manejo de Capas (`Stack`):** Apila componentes en el eje Z en orden de declaración. Su dimensión la determina el hijo no posicionante más grande.
- **Comportamiento `StackFit`:** Controla las restricciones de tamaño ejercidas sobre los hijos no posicionados (`loose`, `expand`, `passthrough`).
- **Posicionamiento Fijo (`Positioned`):** Asigna coordenadas absolutas (`top`, `bottom`, `left`, `right`) a un hijo directo de un `Stack`. `Positioned.fill` expande el widget a todo el contenedor.
- **Alineación Vectorial (`Align`):** Permite ubicar componentes en coordenadas relativas mediante constantes o mediante el par cartesiano `Alignment(x, y)`.
- **Recortado (`clipBehavior`):** La constante `Clip.none` evita que los widgets que sobresalgan de las fronteras físicas del `Stack` sean cortados.
- **Preservación de Estado (`IndexedStack`):** Mantiene todos sus hijos en memoria y solo renderiza visualmente el indicado por la propiedad `index`, conservando intactos los estados de scroll, controladores e insumos de texto.

Bibliografía y Referencias

1. **Google LLC.** (2024). *IndexedStack class documentation - Flutter API*. Recuperado de: <https://api.flutter.dev/flutter/widgets/IndexedStack-class.html>

2. **Google LLC.** (2024). *Stack class documentation - Flutter API*. Recuperado de:
<https://api.flutter.dev/flutter/widgets/Stack-class.html>
3. **Flutter Engineering Team.** (2023). *Architecting Layouts: Understanding Stacks and RenderObjects*.
Recuperado de: <https://docs.flutter.dev/ui/layout>
4. **Windmill, E.** (2020). *Flutter in Action* (Capítulo 4: Complex Layouts and Stacks). Manning Publications.
ISBN: 978-1617296147.